

## Bài 4: Requirements Review – Kiểm duyệt yêu cầu

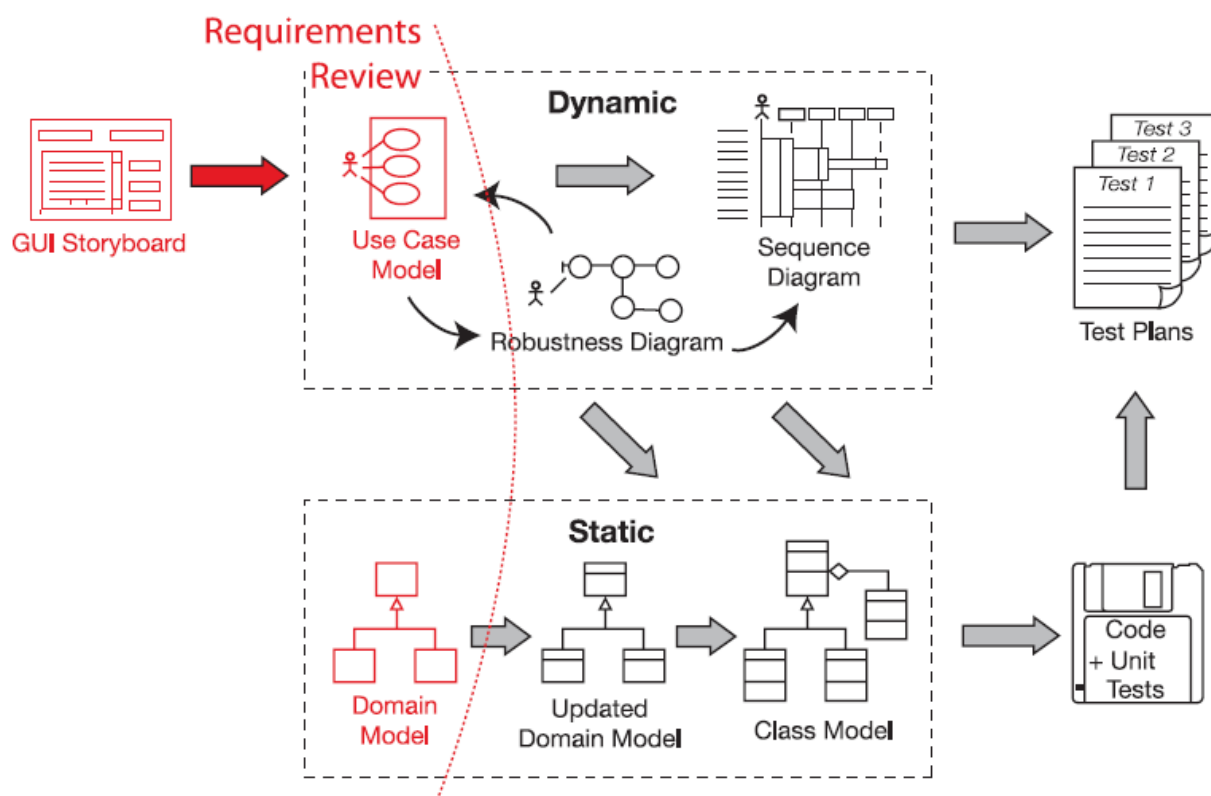
### Nội dung chính

Giải thích rằng Requirements Review là bước kiểm tra và xác thực chất lượng yêu cầu (use case, đặc tả nghiệp vụ, phạm vi hệ thống...) trước khi bắt tay vào thiết kế. ICONIX coi đây là “cửa kiểm soát đầu tiên” để phát hiện các mâu thuẫn, thiếu sót, yêu cầu mơ hồ hoặc không khả thi.

### Mục tiêu cần đạt

- Hiểu được vai trò của Requirements Review trong quy trình ICONIX.
- Biết cách tổ chức một buổi review hiệu quả.
- Nhận diện được các lỗi thường gặp trong tài liệu yêu cầu.
- Áp dụng nguyên tắc review để nâng cao chất lượng đầu vào cho giai đoạn thiết kế.

### 4.1. Giới thiệu



Khi nhóm phát triển đã hoàn thành bước xây dựng *domain model* (Bài 2) và viết xong những bản mô tả *use case* đầu tiên (Bài 3), rất nhiều bạn thường có cảm giác: “Ồ rồi đó, chúng ta có thể bắt tay vào viết code thôi!”. Đây chính là một thời điểm rất nguy hiểm.

Hãy thử tưởng tượng tình huống sau, chắc hẳn nhiều bạn sẽ thấy quen thuộc:

Hỏi: “Dự án của tôi có vẻ rất ổn rồi. Tôi đã vẽ xong domain model, cập nhật trong lúc viết các use case. Các use case cũng đã được viết trong ngữ cảnh của domain model. Thế là đủ để bắt đầu coding rồi đúng không? (Thực ra tôi chưa bao giờ làm kỹ giai đoạn tiền coding như vậy cả!)”

Đáp: “Ừ, gần đúng... nhưng chưa đủ. Khách hàng có thể đã tham gia giai đoạn thu thập yêu cầu ban đầu, và hy vọng là họ cũng có mặt trong buổi workshop mô hình hóa use case của bạn. Nhưng bây giờ, khi thảo luận đầu tiên của các use case đã viết xong, bạn cần mời họ trở lại để cùng kiểm chứng xem mô tả hệ thống hiện tại có thật sự phản ánh đúng những gì họ mong muốn hay không.”

Câu trả lời này nghe có vẻ đơn giản, nhưng thực tế lại chính là điểm mấu chốt: việc kiểm duyệt yêu cầu (*requirements review*) là một bước không thể bỏ qua trong quy trình

ICONIX.

Nếu bạn không làm, hoặc làm một cách qua loa, rất có thể hệ thống cuối cùng sẽ không khớp với những gì khách hàng thật sự cần, dẫn tới hàng loạt rắc rối tốn kém về sau.

## 1.1 Tại sao cần kiểm duyệt yêu cầu?

Rất nhiều người (và cả sinh viên thực tập) thường nghĩ rằng bản mô tả yêu cầu là “tài liệu gửi khách hàng duyệt” chứ không phải là thứ cần phải cùng nhau ngồi lại để rà soát chi tiết. Và thế là họ gửi cho khách hàng một file PDF dài 200 trang, kèm một email lịch sự kiểu:

“Anh/chị vui lòng xem qua, nếu đồng ý thì reply Yes nhé.”

Và rồi, vài phút sau, họ nhận được câu trả lời “Yes, please.” Cảm giác thật dễ chịu... cho tới một năm sau, khi hệ thống hoàn thành, khách hàng kêu trời:

“Trời ơi, đây không phải là thứ tôi muốn xây dựng!”

Đây là một kịch bản rất phổ biến trong các dự án phần mềm lớn. Vấn đề không nằm ở chỗ tài liệu yêu cầu sai ngay từ đầu, mà là do không có một giai đoạn kiểm duyệt có tính cộng tác thật sự giữa khách hàng và nhóm phát triển.

*Requirements review* chính là phiên làm việc đó. Tất cả các bên liên quan — đại diện khách hàng, người dùng cuối, bộ phận marketing, phân tích nghiệp vụ, nhóm kỹ thuật — cùng ngồi lại, mở từng use case, từng mô hình ra xem xét, đặt câu hỏi, chỉnh sửa, thống nhất hiểu biết. Chỉ khi đó, nhóm phát triển mới có thể yên tâm bước vào giai đoạn thiết kế.

## 1.2. Requirements Review trong ICONIX Process

Trong quy trình ICONIX, Requirements Review là một mốc kiểm duyệt (milestone) chính thức, diễn ra sau khi nhóm đã:

- Hoàn thành mô hình miền (*domain model*) ở mức đủ chi tiết (ít nhất 80% các thực thể chính).

- Viết xong bản thảo các use case chính, bao gồm cả luồng chính (basic course) và luồng thay thế (alternate courses).
- Xây dựng các storyboard hoặc bản phác thảo giao diện (nếu có).

Mục tiêu của phiên kiểm duyệt này là:

- Xác nhận rằng mô tả hệ thống hiện tại khớp với yêu cầu thực sự của người dùng.
- Loại bỏ những chỗ hiểu sai, những vùng “mù mờ” trong tài liệu.
- Thống nhất các khái niệm, tên gọi, hành vi giữa các bên tham gia.
- Đảm bảo rằng hệ thống sắp thiết kế sẽ đi đúng hướng ngay từ đầu.

Đây không phải là một buổi thảo luận mang tính hình thức. Nó giống như một “phiên mổ xẻ tập thể”, nơi mà tất cả các thành phần — use case, domain model, storyboard, yêu cầu chức năng — đều được soi kỹ lưỡng để phát hiện điểm bất nhất.

### 1.3. Ai cần tham gia?

Một phiên kiểm duyệt yêu cầu đúng chuẩn thường có các thành phần sau:

- Đại diện khách hàng (customer representative): người có quyền quyết định hoặc hiểu rõ nghiệp vụ.
- Người dùng cuối (end users): những người trực tiếp sử dụng hệ thống (hoặc hệ thống cũ).
- Marketing / quản lý sản phẩm: để đảm bảo các yêu cầu phản ánh đúng nhu cầu thị trường.
- Business analyst / Use case analyst: người chủ trì nội dung use case.
- Đại diện nhóm phát triển (senior developer hoặc architect): để kiểm tra tính khả thi kỹ thuật.
- Người điều phối và thư ký phiên họp: đảm bảo buổi review đi đúng hướng, ghi chép đầy đủ các quyết định.

Việc tập hợp đầy đủ các bên này nghe có vẻ lý tưởng, nhưng thực tế là rất quan trọng. Nếu khách hàng không chịu dành vài giờ để rà soát yêu cầu trước khi ký hợp đồng, thì khả năng cao dự án sẽ rơi vào tình trạng “death march” – chạy nước rút tới cái chết 🤖, bất kể bạn dùng quy trình gì.

### 1.4. Vị trí của Requirements Review trong toàn quy trình

Hãy nhớ rằng ICONIX Process có bốn giai đoạn chính:

#### 1. **Requirements** (Yêu cầu)

2. **Preliminary Design** (Thiết kế sơ bộ)

3. **Detailed Design** (Thiết kế chi tiết)

4. **Implementation** (Cài đặt – lập trình)

Mỗi giai đoạn có một số *milestones* quan trọng. Requirements Review chính là mốc kiểm duyệt đầu tiên, đảm bảo đầu vào cho giai đoạn thiết kế sơ bộ là đúng đắn, đầy đủ, rõ ràng. Nếu bạn làm tốt ở bước này, các bước sau sẽ thuận lợi hơn rất nhiều. Nếu bạn bỏ qua, thì các bước sau sẽ trở thành chuỗi “chữa cháy” liên tục.

## 4.2. Top 10 Requirements Review Guidelines

Sau khi hiểu được lý do tại sao phải tổ chức phiên *requirements review*, bước tiếp theo là nắm thật vững mười nguyên tắc kiểm tra quan trọng. Đây là những “điểm góc” giúp bạn phát hiện sớm lỗi trong use case, domain model và storyboard trước khi bước vào thiết kế.

Các bạn có thể xem mười nguyên tắc này như một checklist khi rà soát tài liệu yêu cầu.

Nếu một trong số chúng bị bỏ qua, khả năng rất cao là hệ thống sau này sẽ gặp những lỗi mơ hồ, sai sót hoặc phát sinh thiết kế phức tạp không cần thiết.

Chúng ta sẽ lần lượt đi qua từng nguyên tắc, giải thích cặn kẽ và minh họa bằng ví dụ Internet Bookstore.

## 10. Domain Model phải bao quát ít nhất 80% các khái niệm miền quan trọng

Nguyên tắc đầu tiên tưởng như đơn giản, nhưng lại là lỗi phổ biến nhất: domain model chưa đủ “già”.

Một domain model tốt phải phản ánh được ít nhất 80% các khái niệm thực tế quan trọng trong bài toán, được mô tả bằng ngôn ngữ phi kỹ thuật mà người dùng cuối hiểu được.

Nếu bạn bỏ sót nhiều thực thể quan trọng, hệ thống thiết kế sẽ không có “xương sống” rõ ràng — dẫn đến các use case trở nên mơ hồ, và việc lập trình sẽ phải “đoán mò” trong lúc coding.

Ví dụ – Internet Bookstore:

Nếu domain model của bạn chỉ có hai lớp là Book và Customer, thì rõ ràng bạn đã thiếu rất nhiều khái niệm như: Order, ShoppingCart, BookReview, RecommendationList, Category, v.v.

Thiếu những đối tượng này thì các use case như *Đặt hàng*, *Gợi ý sách*, *Xem đánh giá* sẽ trở nên cực kỳ mù mờ.

Vì vậy nên:

- Vẽ domain model càng sớm càng tốt.

- Chấp nhận rằng nó sẽ chưa hoàn hảo, nhưng hãy đảm bảo những khái niệm chính đã được thể hiện rõ.
- Đừng sa vào “phân tích tê liệt” (analysis paralysis) để làm nó hoàn mỹ từ đầu — hãy đạt 80% trước, tinh chỉnh dần sau.

## 9. Domain Model phải thể hiện rõ các quan hệ is-a và has-a

Quan hệ is-a (tổng quát hoá, *generalization*) và has-a (kết tập, *aggregation*) giúp domain model trở nên cụ thể và có cấu trúc thay vì chỉ là danh sách các hộp rời rạc.

- *is-a* cho thấy một đối tượng là một trường hợp cụ thể của một lớp khái quát hơn (VD: BestsellerList *is a* BookList).
- *has-a* cho thấy mối quan hệ chứa đựng, thành phần (VD: ShoppingCart *has a* danh sách BookItem).

Ví dụ – Internet Bookstore:

Lớp BookList có thể có nhiều dạng cụ thể như: WishList, RecommendationList, BestsellerList.

Mỗi Customer *has-a* ShoppingCart.

Order *has-a* nhiều OrderLineItem.

Nếu bạn không biểu diễn rõ những mối quan hệ này, sơ đồ domain sẽ trở nên mơ hồ, khó đọc và khó liên kết với use case sau này.

Vì vậy:

- Đừng quá quan trọng hoá chi tiết giữa aggregation và composition ở giai đoạn này.
- Tập trung làm rõ các mối quan hệ chính để nhóm phát triển có ngôn ngữ chung khi bàn về thiết kế.

## 8. Use Case phải mô tả đầy đủ cả luồng chính và luồng thay thế, ở thể chủ động

Một use case đúng chuẩn không chỉ mô tả “điều bình thường xảy ra” (*sunny day scenario*) mà còn phải bao gồm các trường hợp thay thế / ngoại lệ (*rainy day scenarios*).

Ví dụ – Internet Bookstore:

Với use case *Xem chi tiết sách (Show Book Details)*:

- Luồng chính (basic course): khách hàng truy cập trang chủ, chọn một cuốn sách, hệ thống hiển thị chi tiết sách.
- Luồng thay thế (alternate): khách hàng chọn một sách không tồn tại → hệ thống hiện trang “Book not found”.

Nếu bạn chỉ viết luồng chính mà quên alternate, hệ thống sẽ không có cách xử lý khi xảy ra trường hợp lỗi hoặc ít gặp — và đó chính là nơi các bug thực tế nảy sinh.

## 7. Không trộn lẫn yêu cầu chức năng (functional requirements) với use case

Đây là lỗi rất phổ biến ở sinh viên và cả các nhóm dự án non kinh nghiệm: Họ viết vào use case những câu kiểu như “Hệ thống phải hỗ trợ 300 giao dịch/giây” hoặc “Hệ thống sẽ tự động gửi email xác nhận”. Đây là yêu cầu chức năng ở thể bị động (*shall statements*), không phải là hành vi mô tả tương tác.

Khi bạn trộn hai loại này vào nhau, văn bản trở nên rối rắm và khó hiểu.

- Use case → mô tả tương tác người dùng – hệ thống (hành vi).
- Functional requirement → mô tả năng lực hệ thống (điều kiện cần đạt).

## 6. Tổ chức các Use Case thành các gói (packages)

Trong các hệ thống lớn, số lượng use case có thể lên tới hàng trăm. Nếu bạn để tất cả chúng lộn xộn trong một sơ đồ duy nhất, bạn sẽ nhanh chóng rơi vào hỗn loạn.

**Giải pháp:** chia use case thành **các gói chức năng (package)** theo tiêu chí:

- Theo nhóm chức năng (VD: Quản lý người dùng, Xử lý đơn hàng, Xem sách...)
- Theo phiên bản phát hành (release)
- Hoặc kết hợp cả hai.

**Ví dụ – Internet Bookstore:**

- Gói “Browsing” → *Search Books, Show Book Details, List Bestsellers*
- Gói “Shopping” → *Add to Cart, Checkout, Apply Discount Code*
- Gói “Account” → *Login, Register, View Order History*

## 5. Viết Use Case trong ngữ cảnh của Domain Model

Đây là một nguyên tắc quan trọng giúp tránh **mơ hồ ngữ nghĩa**. Nếu use case chỉ viết theo ngôn ngữ người dùng (user terms) mà không tham chiếu rõ ràng đến domain model, người đọc sẽ dễ hiểu lệch.

Ví dụ – Internet Bookstore: Thay vì viết:

“Người dùng xem thông tin sách.”

Hãy viết:

“Customer chọn một Book trong BestsellerList. Hệ thống hiển thị BookDetails.”

Khi đó, lập trình viên sẽ biết chính xác các lớp nào liên quan, thiết kế nào cần chuẩn bị.

## 4. Viết Use Case trong ngữ cảnh của Giao diện người dùng (User Interface)

Bên cạnh domain model, giao diện người dùng cũng là một “điểm neo” quan trọng giúp tránh mơ hồ.

Ví dụ – Internet Bookstore:

Thay vì viết mơ hồ “Khách hàng chọn sách để xem chi tiết”, bạn nên viết:

“Customer nhấp vào liên kết (link) của một Book trên màn hình BestsellerList. Hệ thống tải dữ liệu và hiển thị BookDetailsScreen.”

### Tips:

- Đặt tên cụ thể cho các màn hình và sử dụng chúng xuyên suốt trong use case.
- Không cần tạo giao diện thật, nhưng nên có phác thảo rõ ràng.
- Điều này giúp nhóm hiểu nhanh “chuyện gì xảy ra ở đâu” khi thực hiện hành vi.

## 3. Bổ sung Use Case bằng storyboard, bản phác giao diện hoặc nguyên mẫu GUI

Hình ảnh luôn giúp dễ hiểu hơn chữ. Một bản storyboard hay bản phác giao diện đơn giản sẽ giúp bạn:

- Hiểu rõ hành vi người dùng trên từng màn hình.
- Tránh quên các nút bấm quan trọng (ví dụ: nút *Cancel*).
- Nhận ra các luồng tương tác bị bỏ sót.

### Ví dụ – Internet Bookstore:

Một bản phác đơn giản cho trang *Book Details* với nút *Write Review*, *Add to Cart*, *Back to List* sẽ giúp bạn rà soát toàn bộ hành vi trên trang đó.

### Tips:

- Không cần làm prototype hoàn chỉnh.
- Chỉ cần rõ ràng, nhanh gọn, tập trung vào hành vi tương tác.
- Nếu có prototype thật, hãy trình bày nó trong phiên review để mọi người cùng “walk through” hành vi.

## 2. Rà soát Use Case với đúng nhóm người

Một lỗi cơ bản nhưng tai hại: review mà không có người dùng thật sự hoặc khách hàng.

Nếu chỉ có lập trình viên ngồi với nhau, họ sẽ đồng ý với nhau... nhưng đó không đảm bảo rằng yêu cầu đúng 😊.

Một phiên kiểm duyệt tốt phải có:

- Người dùng cuối thật sự (đưa phản hồi thực tế).
- Đại diện khách hàng (xác nhận nghiệp vụ).
- Business analyst (giữ logic use case).
- Nhóm kỹ thuật (đảm bảo khả năng thực thi).

Tips:

- Cần có người điều phối để tránh sa đà vào chi tiết kỹ thuật sớm.
- Cần có người ghi chép quyết định, action items rõ ràng.
- Đây không phải buổi thuyết trình một chiều mà là workshop cộng tác.

## 1. Cấu trúc phiên kiểm duyệt theo “Tám bước rà soát Use Case”

Nguyên tắc cuối cùng chính là khung xương cho toàn bộ phiên kiểm duyệt: Eight Easy Steps to a Better Use Case.

Phần này sẽ được trình bày chi tiết ở phần tiếp theo, nhưng tạm thời, bạn cần nhớ rằng:

- Chúng ta không ngồi đọc từng dòng một cách ngẫu hứng.
- Chúng ta đi theo một trình tự cụ thể gồm 8 bước, giúp nhanh chóng phát hiện và sửa lỗi từ lớn đến nhỏ.
- Đây là “bí quyết” khiến các phiên review ICONIX có hiệu quả cao và tiết kiệm thời gian.

### 4.3. Tám bước rà soát Use Case

Sau khi nhóm đã xây dựng được *domain model* và viết bản thảo các *use case*, bước kiểm duyệt yêu cầu (Requirements Review) sẽ không chỉ dừng lại ở việc “đọc cho vui”



Chúng ta cần một **trình tự cụ thể, rõ ràng** để rà soát từng use case, phát hiện và chỉnh sửa những điểm mơ hồ, thiếu chính xác hoặc thiếu liên kết với mô hình.

ICONIX đề xuất một phương pháp rất thực tế gọi là: “**Eight Easy Steps to a Better Use Case**” — Tám bước đơn giản để biến một use case tệ thành một use case tốt.

Các bạn hãy nhớ rằng: đây không chỉ là một “mẹo nhỏ”, mà là một **khung kiểm duyệt có hệ thống**, được đúc kết từ kinh nghiệm của rất nhiều dự án lớn. Chúng ta sẽ đi lần lượt từng bước, giải thích tại sao bước đó quan trọng, và minh họa bằng ví dụ thực tế với use case “Show Book Details” trong hệ thống Internet Bookstore.

## Bước 1: Loại bỏ tất cả những gì nằm ngoài phạm vi (Out of Scope)

Rất nhiều bạn khi viết use case thường có xu hướng “kể lể” quá nhiều. Họ bắt đầu viết từ những hành động xảy ra *trước* khi use case thật sự bắt đầu, hoặc kết thúc bằng những tình huống *sau* khi use case kết thúc. Kết quả là nội dung trở nên dài dòng, lẫn lộn giữa nhiều chức năng khác nhau.

Ví dụ – Show Book Details: Một bản nháp điển hình có thể bắt đầu như sau:

“Nếu người dùng có tài khoản, họ có thể đăng nhập trước. Sau đó, họ đi tới trang chủ. Trang chủ phải dễ truy cập và có khu vực tìm kiếm. Sau đó họ xem thông tin chi tiết sách, và nếu muốn, họ có thể mua sách bằng Visa, Amex hoặc séc.”

Phần “đăng nhập” và “mua sách” rõ ràng là nằm ngoài phạm vi của use case *Xem chi tiết sách*. Chúng thuộc về use case *Login* và *Purchase Book*.

Khi rà soát, bước đầu tiên là cắt bỏ những đoạn này, để giữ cho use case tập trung đúng phạm vi chức năng. Mục tiêu là làm cho use case ngắn gọn, tập trung, tránh kéo theo các luồng hành vi không liên quan.

## Bước 2: Chuyển toàn bộ câu bị động sang thể chủ động (Passive → Active)

Một lỗi kinh điển trong viết use case là sử dụng câu bị động (passive voice), kiểu như:

“Trang chủ sẽ được hiển thị.”

“Chi tiết sách sẽ được lấy ra.”

“Hệ thống sẽ được yêu cầu xác thực người dùng.”

Những câu này không thể hiện rõ ai làm gì và phản ứng ra sao. Trong khi đó, bản chất của use case là mô tả chuỗi tương tác hành động – phản hồi giữa người dùng và hệ thống.

Ví dụ – Show Book Details: Thay vì viết:

“Chi tiết sách sẽ được hiển thị.”

Hãy viết:

“Customer nhấp vào liên kết của cuốn Book. Hệ thống tải dữ liệu và hiển thị Book Details screen.”

Mục tiêu:

- Làm rõ “ai hành động trước” và “hệ thống phản hồi thế nào”.
- Giúp lập trình viên dễ dàng chuyển hóa use case thành thiết kế và code.

*Tips:* Đổi câu bị động sang chủ động càng sớm, việc rà soát ở các bước sau càng dễ. Trong nhiều dự án thực tế, chỉ riêng bước này đã giúp “lột xác” một use case từ mơ hồ thành rõ ràng.

### Bước 3: Kiểm tra xem Use Case có quá trừu tượng hay không

Nhiều tài liệu hướng dẫn use case kiểu truyền thống thường khuyên:

“Hãy viết use case một cách trừu tượng, công nghệ độc lập, không ràng buộc giao diện.”

Nghe có vẻ đúng về lý thuyết, nhưng thực tế là:

“Use case trừu tượng” thường đồng nghĩa với “mơ hồ, không thể thiết kế được” 😊

Ví dụ – Show Book Details: Một câu kiểu như:

“Khách hàng duyệt chi tiết sách. Hệ thống hiển thị thông tin.”

Câu này bỏ qua toàn bộ hành vi thực tế: Khách hàng làm gì để “duyet”? → Nhấp link? Gõ ISBN? Quét QR? Hệ thống “hiển thị thông tin” cụ thể là màn hình nào? Gồm những thành phần nào?

Khi rà soát, hãy yêu cầu người viết mô tả cụ thể hành động và phản ứng, theo đúng mô hình “User action → System response”.

Phiên bản tốt hơn:

“Customer nhấp vào liên kết của Book trong Bestseller List. Hệ thống truy xuất Book details và hiển thị Book Details screen.”

Mục tiêu:

- Loại bỏ sự mơ hồ, khái quát quá mức.
- Tăng độ chính xác để có thể chuyển thẳng sang thiết kế giao diện & tương tác.

### Bước 4: Phản ánh chính xác giao diện người dùng (GUI)

Một use case tốt phải phản ánh đúng cách người dùng tương tác với giao diện. Điều này đòi hỏi nhóm phát triển phải đặt tên rõ ràng cho các màn hình và sử dụng những tên này trong use case.

Ví dụ – Show Book Details: Thay vì viết:

“Khách hàng chọn sách để xem chi tiết.”

Hãy viết:

“Customer nhấp vào liên kết của một Book trên màn hình Bestseller List. Hệ thống hiển thị Book Details Screen.”

Nếu bạn không phản ánh rõ giao diện, rất dễ xảy ra hiện tượng “mỗi người tưởng tượng một kiểu”:

- BA nghĩ tới một trang HTML đơn giản
- UX designer tưởng tượng một modal popup

- Developer thì thiết kế một ứng dụng desktop 🤖

Mục tiêu:

- Đồng nhất cách hiểu giữa các bên.
- Đảm bảo thiết kế sau này khớp với hành vi mô tả trong use case.

### Bước 5: Đặt tên rõ ràng cho các Domain Object tham gia

Một use case không thể tách rời domain model. Nếu phần mô tả không nhắc đến các domain object cụ thể, lập trình viên sẽ không biết chính xác mình đang thao tác với thực thể nào.

Ví dụ – Show Book Details: Thay vì viết:

“Người dùng truy cập khu vực chính của website và xem thông tin sách.”

Hãy viết:

“Customer truy cập Bookstore’s main area, chọn một Book trong BestsellerList. Hệ thống hiển thị BookDetails.”

Ở đây, Customer, Bookstore, Book, BestsellerList, BookDetails đều là các domain object có thật trong mô hình. Việc dùng tên chính xác giúp loại bỏ cách hiểu mơ hồ, tránh việc tạo ra những lớp trùng lặp hoặc sai nghĩa khi thiết kế.

Mục tiêu:

- Liên kết rõ ràng giữa use case ↔ domain model.
- Đảm bảo ngữ nghĩa chính xác khi chuyển sang thiết kế hướng đối tượng.

### Bước 6: Đảm bảo đã có đầy đủ các luồng thay thế (Alternate Courses)

Hầu hết độ phức tạp của hệ thống nằm ở các luồng thay thế, chứ không phải luồng chính.

Một use case không có alternate course gần như chắc chắn là chưa được phân tích kỹ.

Ví dụ – Show Book Details: Alternate Course cơ bản:

“Book không tồn tại → Hệ thống hiển thị trang Book Details Not Found.”

Nếu không có bước này, nhóm phát triển sẽ không thiết kế các tình huống lỗi, không kiểm thử đầy đủ, và hệ thống sẽ... crash trong thực tế.

Mục tiêu:

- Bao quát đủ các trường hợp không thuận lợi.
- Giảm thiểu bug phát sinh từ những “góc khuất” không được nghĩ tới.

Tips: Nếu bạn review một use case mà không thấy alternate course nào → hãy bật “chuông báo động” .

## Bước 7: Truy vết từng yêu cầu về use case tương ứng

Bước này thường bị bỏ qua trong các nhóm nhỏ, nhưng lại cực kỳ quan trọng trong các dự án lớn, hoặc khi có hợp đồng ràng buộc. Mục tiêu là đảm bảo rằng mỗi yêu cầu chức năng đều có ít nhất một use case thực hiện nó.

Ví dụ – Show Book Details: Yêu cầu:

“Người dùng phải có thể xem đánh giá và chấm sao của sách.”

Nếu use case *Show Book Details* không nhắc đến phần hiển thị đánh giá, bạn biết ngay rằng yêu cầu này đang bị “rơi rớt”.

Trong công cụ Enterprise Architect, người ta thường dùng traceability matrix để biểu diễn mối liên hệ này. Ở quy mô nhỏ, bạn có thể lập bảng Excel đơn giản:

| Requirement       | Use Case              |
|-------------------|-----------------------|
| View book ratings | Show Book Details     |
| Write review      | Write Customer Review |

Mục tiêu:

- Đảm bảo tính đầy đủ của tài liệu yêu cầu.
- Là cơ sở kiểm tra nghiệm thu sau này.

## Bước 8: Đảm bảo Use Case mô tả rõ người dùng đang cố gắng làm gì

Bước cuối cùng — nhưng không kém phần quan trọng — là kiểm tra xem toàn bộ use case có đang tập trung vào mục tiêu của người dùng hay không.

Một use case tốt không phải là bản mô tả các chi tiết kỹ thuật hệ thống, mà là: “Câu chuyện về cách người dùng đạt được mục tiêu của mình thông qua hệ thống.”

Ví dụ – Show Book Details: Mục tiêu người dùng:

“Xem thông tin chi tiết của một cuốn sách cụ thể.”

Nếu use case bắt đầu lạc sang đăng nhập, thống kê, quảng cáo, thanh toán... nghĩa là nó đã lệch khỏi mục tiêu ban đầu.

Mục tiêu:

- Giữ cho use case tập trung, gọn gàng.
- Tránh lạc đề sang các hành vi không liên quan.

## 4.4. Use Case “Show Book Details”

Sau khi chúng ta đã nắm vững mười nguyên tắc kiểm duyệt yêu cầu và tám bước rà soát use case, giờ là lúc xem thử quy trình này vận hành trong thực tế như thế nào.

Phần này sẽ mô phỏng một buổi kiểm duyệt thực tế cho use case: “Show Book Details” – Xem chi tiết sách

### Bối cảnh phiên kiểm duyệt

Để phiên kiểm duyệt hiệu quả, bạn cần có những người sau tham gia:

- Reviewer: người kiểm duyệt giàu kinh nghiệm (thường là kiến trúc sư hệ thống hoặc mentor quy trình).
- Analyst: người chịu trách nhiệm viết use case (thường là business analyst hoặc người thiết kế chức năng).
- Có thể có thêm: đại diện người dùng, khách hàng, trưởng nhóm kỹ thuật, hoặc marketing.

Để tránh phiên họp quá đông, ví dụ minh họa này chỉ tập trung vào cuộc đối thoại giữa Reviewer và Analyst — hai nhân vật chính.

### Bước 1: Loại bỏ những nội dung ngoài phạm vi

#### Analyst:

“Đây là bản use case ‘Show Book Details’ mà tôi viết:

- Nếu người dùng có tài khoản, họ có thể đăng nhập trước (dù không bắt buộc).
- Sau đó họ truy cập trang chính của website.
- Trang chính phải dễ truy cập, có vùng tìm kiếm và danh mục như top 10 sách bán chạy, bình chọn của biên tập viên, v.v.
- Sau đó họ xem thông tin chi tiết của sách.
- Nếu muốn, họ có thể mua sách bằng Visa, Amex hoặc séc (séc phải được nhận trước khi gửi hàng).”

**Reviewer** (lắm bầm 🤔):

“Ôi trời ơi... lại một use case kiểu ‘tất cả trong một’.”

#### Reviewer:

“Đầu tiên, phần đăng nhập và phần mua sách rõ ràng không thuộc phạm vi của use case này.

Chúng phải là use case khác: *Login* và *Purchase Book*.

Việc bạn gộp chúng vào chỉ làm rối nội dung.”

Sau khi loại bỏ các phần thừa, luồng cơ bản còn lại chỉ là:

- Truy cập trang chính → xem danh mục sách → chọn một cuốn sách → xem thông tin chi tiết.

Đây là Bước 1 trong tám bước: giữ cho use case tập trung đúng phạm vi.

## Bước 2: Chuyển từ câu bị động sang câu chủ động

Phiên bản gốc của Analyst chứa rất nhiều câu bị động:

“Trang chính phải dễ truy cập và có vùng tìm kiếm.”

“Thông tin sách sẽ được hiển thị.”

Những câu này không mô tả rõ hành vi tương tác.

Reviewer:

“Hãy viết lại theo đúng mô hình:

*Khách hàng hành động → Hệ thống phản hồi.*

Ví dụ: ‘Customer nhấp vào liên kết của một Book. Hệ thống hiển thị Book Details screen.’”

Sau khi sửa, đoạn mô tả trở nên cụ thể và rõ ràng hơn:

“Customer truy cập trang chính, chọn một Book từ danh sách, hệ thống hiển thị màn hình Book Details.”

Đây là Bước 2: chuyển sang thể chủ động để làm rõ luồng tương tác.

## Bước 3: Loại bỏ tính trừu tượng

Câu kiểu như:

“Khách hàng duyệt thông tin sách.” → rất mơ hồ. Duyệt bằng cách nào? Giao diện ra sao? Hệ thống phản ứng thế nào?

**Reviewer:**

“Bạn phải mô tả chính xác hành động. Ví dụ:

‘Customer nhấp vào liên kết của cuốn Book trong Bestseller List. Hệ thống truy xuất Book details và hiển thị Book Details screen.’”

Analyst:

“À, đúng rồi, tôi viết kiểu khái quát quá.”

Đây là Bước 3: loại bỏ sự trừu tượng, viết cụ thể hành vi.

## Bước 4: Phản ánh đúng giao diện

Reviewer:

“Bạn cần nhắc rõ các màn hình giao diện. Ví dụ: ‘Bestseller List screen’ và ‘Book Details screen’. Nếu bạn không nêu rõ, mỗi người sẽ tưởng tượng một giao diện khác nhau — và điều đó dẫn đến thiết kế không thống nhất.”

Analyst:

“Vâng, tôi sẽ đặt tên màn hình rõ ràng và sử dụng xuyên suốt.”

Đây chính là cách đảm bảo mô tả hành vi phù hợp với thiết kế UI sau này.

## Bước 5: Liên kết với Domain Model

Khi đọc use case, Reviewer nhận ra Analyst viết “user” và “website” trong khi domain model định nghĩa là Customer và Bookstore.

Reviewer:

“Nếu bạn dùng thuật ngữ không khớp với domain model, sau này khi thiết kế lớp, chắc chắn sẽ xảy ra hiểu nhầm hoặc tạo class trùng lặp.

Ở đây, bạn phải viết rõ ‘Customer’, ‘Bookstore’, ‘BestsellerList’, ‘BookDetails’ đúng theo domain model.”

Sau khi sửa, bản mô tả trở nên chính xác về ngữ nghĩa:

“Customer truy cập Bookstore’s main area, chọn một Book từ BestsellerList. Hệ thống hiển thị Book Details.”

**Chốt lại ở bước này:** dùng tên domain object đúng chuẩn để kết nối chặt chẽ giữa use case và mô hình miền.

## Bước 6: Bổ sung luồng thay thế (Alternate Courses)

Phiên bản gốc của Analyst **không có alternate course nào** — một dấu hiệu nguy hiểm 🙄.

Reviewer hỏi:

“Nếu sách không tồn tại thì sao?”

**Analyst:**

“Ờ... chưa nghĩ đến trường hợp đó...”

**Reviewer:**

“Đây chính là lý do cần alternate course. Ví dụ:

‘Book not found: Hệ thống hiển thị Book Details Not Found screen.’”

Alternate course không chỉ giúp hệ thống “đứng vững” trước tình huống lỗi, mà còn giúp thiết kế và kiểm thử bao quát đầy đủ.

## Bước 7: Truy vết yêu cầu (Requirements Traceability)

Reviewer mở danh sách yêu cầu và chỉ ra:

“Yêu cầu số 11: *Hệ thống phải cho phép người dùng xem đánh giá và xếp hạng sao của sách.*”

Use case này phải thể hiện điều đó, vì đây là chức năng chính của màn hình Book Details.”

Nếu use case không thể hiện, đó là một **lỗ hổng truy vết yêu cầu** — sau này dễ dẫn tới thiếu chức năng khi nghiệm thu.

Analyst:

“Đúng rồi, tôi sẽ bổ sung phần hiển thị đánh giá vào luồng chính.”

Tóm lại: đảm bảo mỗi yêu cầu chức năng được phản ánh qua ít nhất một use case.

## Bước 8: Đảm bảo tập trung vào mục tiêu người dùng

Mục tiêu của use case *Show Book Details* là:

“Cho phép khách hàng xem thông tin chi tiết của một cuốn sách.”

Nếu trong bản mô tả có các đoạn rẽ sang đăng nhập, thống kê, quảng cáo, thanh toán...

→ là dấu hiệu lệch mục tiêu.

Reviewer:

“Use case của bạn phải tập trung vào hành vi ‘xem chi tiết sách’, không lan man sang các mục tiêu khác.” → giữ cho use case tập trung đúng mục tiêu nghiệp vụ cốt lõi.

## Kết quả sau khi kiểm duyệt

Sau khi áp dụng 8 bước, đoạn mô tả *Show Book Details* trở nên rõ ràng, chính xác, và có thể chuyển sang thiết kế trực tiếp:

### BASIC COURSE:

Customer truy cập Bookstore’s main area, chọn một Book trong BestsellerList. Hệ thống truy xuất Book details và hiển thị Book Details screen, bao gồm tiêu đề, mô tả, giá, và các đánh giá của khách hàng.

### ALTERNATE COURSE:

Book không tồn tại → Hệ thống hiển thị Book Details Not Found screen.

Phiên bản này ngắn gọn, súc tích, tập trung vào hành vi, và nhất quán với domain model & UI.

## 4.5. Truy vết yêu cầu

Trong các dự án phần mềm thực tế — đặc biệt là các dự án lớn, có hợp đồng rõ ràng và nhiều bên liên quan — việc truy vết yêu cầu (requirements traceability) là một bước vô cùng quan trọng nhưng thường bị đánh giá thấp.

Nói ngắn gọn, truy vết yêu cầu có nghĩa là: Xác định và duy trì mối liên kết hai chiều giữa từng yêu cầu (requirement) và những thành phần khác trong hệ thống như:

- Các use case (tình huống sử dụng)
- Các thiết kế (lớp, thành phần)
- Các đoạn mã cài đặt
- Các kịch bản kiểm thử (test cases)

Mục đích chính là để bảo đảm rằng mọi yêu cầu được phản ánh đầy đủ trong hệ thống, từ giai đoạn phân tích cho tới triển khai và kiểm thử.

Giả sử trong tài liệu yêu cầu của hệ thống Internet Bookstore có đoạn sau:

**Requirement 11:** “Hệ thống phải cho phép người dùng xem đánh giá (review) và xếp hạng (rating) của các sách.

Các đánh giá phải được kiểm duyệt bởi nhân viên trước khi công bố.

Các đánh giá dài phải được rút gọn trên trang Book Details, với liên kết để xem toàn bộ trên trang riêng.”

Khi tiến hành kiểm duyệt use case Show Book Details, Reviewer nhận thấy rằng:

- Use case hiện tại chưa nhắc gì đến phần đánh giá, rating.
- Điều này có nghĩa là Requirement 11 chưa được phản ánh → Một lỗ hổng truy vết yêu cầu.

Sau khi rà soát, Analyst bổ sung nội dung tương ứng vào use case:

“Hệ thống hiển thị tiêu đề, mô tả, giá, và các đánh giá đã được duyệt của sách, bao gồm điểm xếp hạng trung bình.”

Khi đó, bảng truy vết có thể trông như sau:

| Requirement                      | Use Case liên quan                       |
|----------------------------------|------------------------------------------|
| #1 – Bookstore web-based         | Áp dụng cho tất cả                       |
| #11 – View reviews and ratings   | Show Book Details, Write Customer Review |
| #15 – Moderator approves reviews | Moderate Reviews                         |

Đây là cách đơn giản nhưng hiệu quả để bảo đảm rằng mọi yêu cầu được “gắn” vào ít nhất một use case.

Trong công cụ Enterprise Architect, điều này thường được thực hiện bằng *traceability matrix*. Ở các nhóm nhỏ, bạn có thể dùng Excel hoặc bảng Markdown đơn giản cũng đủ.

### 4.5.1 Nếu không truy vết, điều gì xảy ra?

Nếu bạn không thực hiện truy vết yêu cầu một cách có hệ thống, những rủi ro sau sẽ xảy ra:

- **Thiếu chức năng khi nghiệm thu:** Một số yêu cầu “rơi rớt” không được thực hiện → khách hàng không hài lòng, dự án bị kéo dài.
- **Không thể chứng minh đã đáp ứng yêu cầu hợp đồng:** Điều này đặc biệt nghiêm trọng khi dự án có ràng buộc pháp lý hoặc thanh toán theo hợp đồng.
- **Kiểm thử không đầy đủ:** Nếu không biết yêu cầu nào gắn với use case nào, bạn sẽ không biết kiểm thử cái gì.
- **Khó bảo trì và nâng cấp:** Khi yêu cầu thay đổi, bạn không biết chúng ảnh hưởng đến use case hoặc thành phần nào → dễ gây lỗi dây chuyền.

Truy vết yêu cầu không chỉ là “thủ tục hành chính” mà là **một cơ chế đảm bảo chất lượng** xuyên suốt vòng đời phát triển phần mềm.

### 4.5.2 Vai trò của Requirements Review trong toàn quy trình ICONIX

Ở cuối chương này, chúng ta cần nhìn Requirements Review trong bức tranh tổng thể của ICONIX Process:

Requirements → Preliminary Design → Detailed Design → Implementation

- Requirements Review là mốc kiểm duyệt đầu tiên (checkpoint #1).
- Nó đảm bảo rằng đầu vào cho thiết kế sơ bộ là chính xác, rõ ràng, không mơ hồ.
- Đây là nơi tất cả các use case, domain model, storyboard, yêu cầu... được rà soát kỹ lưỡng, thống nhất giữa các bên.
- Nếu bước này được thực hiện tốt, các giai đoạn sau sẽ trôi chảy và tiết kiệm thời gian đáng kể.
- Ngược lại, nếu bỏ qua hoặc làm qua loa, toàn bộ dự án có thể phải trả giá rất đắt khi phát hiện lỗi ở giai đoạn coding hoặc kiểm thử.

Bạn có thể hình dung Requirements Review như một cửa kiểm định chất lượng đầu tiên.

Một khi tài liệu yêu cầu “lọt” qua cửa này, nó sẽ trở thành nền tảng cho thiết kế, code và test. Vì vậy, việc kiểm duyệt kỹ ngay từ đây có giá trị cực lớn.

## Kết luận

Requirements Review là mốc kiểm duyệt đầu tiên và quan trọng trong quy trình ICONIX, giúp đảm bảo yêu cầu hệ thống rõ ràng, chính xác và thống nhất trước khi bước sang thiết kế. Thông qua phiên kiểm duyệt, nhóm phát triển và các bên liên quan cùng rà soát domain model, use case và storyboard để phát hiện sớm những điểm

mơ hồ hoặc thiếu sót. Mười nguyên tắc kiểm duyệt cung cấp khung đánh giá chất lượng tài liệu, trong khi Tám bước rà soát use case giúp chuyển hóa một bản thảo rời rạc thành mô tả hành vi rõ ràng, có thể thiết kế và lập trình. Ví dụ “Show Book Details” minh họa cách áp dụng các bước này trong thực tế, từ việc loại bỏ nội dung ngoài phạm vi, viết thể chủ động, bổ sung alternate courses, đến truy vết yêu cầu. Thực hiện Requirements Review nghiêm túc giúp giảm sai sót, tránh hiểu nhầm, tiết kiệm chi phí sửa lỗi và đảm bảo tính nhất quán cho toàn bộ dự án. Đây chính là nền móng vững chắc cho các giai đoạn thiết kế và triển khai sau này.

## Câu hỏi ôn tập

1. Tại sao Requirements Review được coi là một mốc kiểm duyệt quan trọng trong quy trình ICONIX?
2. Những bên nào nên tham gia phiên kiểm duyệt yêu cầu? Vì sao?
3. Hãy liệt kê và giải thích ngắn gọn 5 trong số 10 nguyên tắc kiểm duyệt yêu cầu.
4. Tại sao phải viết use case ở thể chủ động? Việc này giúp ích gì trong thiết kế và lập trình?
5. Alternate Course là gì? Cho ví dụ cụ thể với use case “Show Book Details”.
6. Mục đích chính của bước “Truy vết yêu cầu” là gì? Hãy mô tả cách thực hiện đơn giản nhất.
7. Hãy trình bày ngắn gọn 8 bước rà soát use case và vai trò của chúng.
8. Điều gì có thể xảy ra nếu nhóm dự án bỏ qua hoặc làm sơ sài bước Requirements Review?
9. Trong ví dụ Show Book Details, lỗi lớn nhất của Analyst ban đầu là gì?
10. Tại sao Requirements Review lại đặc biệt quan trọng trong các dự án có hợp đồng ràng buộc?